P08 — Simulador de partículas con física básica

Lenguaje de Programación 1 (Imperativo)

Grupo 1

Tom Jordan Pérez López, Raul Antonio Guzmán López, Ricardo Gaspar Reyes Campos y Sandro Giancarlo Chavez Vicaña

Facultad de Ingeniería Industrial y de Sistemas, Universidad Nacional de Ingeniería

Ing. Juan Luis Herencia Guerra

Abril, 2026

Tabla de contenido

Abstract..... 2

Análisis y Diseño..... 3

Requerimientos Funcionales..... 3

Requerimientos no funcionales..... 5

Restricciones del Sistema..... 6

Descripción de las Ecuaciones Físicas..... 7

Integración de Movimiento (Método de Euler Semi-Implícito)..... 7

Gravedad (Aceleración Constante)..... 7

Fricción (Amortiguamiento Viscoso)..... 8

Colisión Elástica entre Partículas..... 8

Colisión con Paredes (Reflexión y Restitución)..... 10

Análisis de Energía del Sistema..... 11

Diseño de las Estructuras y Algoritmos de Simulación..... 13

Variables Adicionales..... 13

Parámetros físicos del sistema..... 13

Parámetros numéricos de simulación..... 13

Parámetros del dominio espacial..... 14

Diseño del Struct de Partícula..... 14

Descripción de los Atributos..... 15

Bucle de Simulación..... 16

Flujo lógico por cada tick:.....16

Análisis algorítmico.....20

Referencias.....21

Resumen

En desarrollo.

Palabras clave: partícula, colisión, tick.

Análisis y Diseño

Requerimientos Funcionales

Tabla 1. Requerimientos funcionales del sistema.

Requerimientos Funcionales		
1	Inicialización de partículas	El sistema debe inicializar N partículas con posiciones aleatorias dentro del espacio de simulación 2D, sin solapamiento inicial. Cada partícula tendrá: posición (x, y), velocidad (vx, vy), masa y radio configurables o generados aleatoriamente dentro de rangos definidos. Además, cada tick hace referencia a la unidad de paso de tiempo (por ejemplo, ns, ms, s, etc.).
2	Integración de movimiento (Euler)	El simulador debe actualizar la posición de cada partícula en cada tick mediante integración de Euler hacia adelante (forward Euler), usando el paso de tiempo dt como parámetro configurable.
3	Aplicación de gravedad	El simulador debe aplicar una aceleración gravitacional constante g (configurable, por defecto 9.81 m/s²) sobre el eje vertical (vy) de cada partícula en cada tick.
4	Aplicación de fricción	El simulador debe aplicar un coeficiente de amortiguamiento viscoso (fricción) sobre ambas componentes de velocidad en cada tick, reduciendo gradualmente la energía cinética del sistema.
5	Detección de	El simulador debe detectar colisiones entre todos los pares posibles

	colisiones entre partículas	de partículas usando comparación de distancias euclidianas versus la suma de sus radios. Se deben evaluar N·(N−1)/2 pares únicos por tick.
6	Resolución de colisiones elásticas en dos dimensiones	Al detectarse una colisión entre dos partículas, el sistema debe resolver la respuesta usando las fórmulas de colisión elástica 2D: descomposición en componentes normal y tangencial, transferencia de momento y corrección de solapamiento (overlap correction).
7	Colisión con paredes	El simulador debe detectar cuando una partícula alcanza o sobrepasa los límites del espacio (ancho × alto) y aplicar reflexión con coeficiente de restitución configurable (0 = inelástico, 1 = perfectamente elástico).
8	Cálculo de energía del sistema	El simulador debe calcular en cada tick la energía cinética total (Ek), la energía potencial gravitacional total (Ep) y la energía mecánica total (Et = Ek + Ep) de todas las partículas.
9	Exportación a CSV	El simulador debe generar tres archivos CSV al finalizar la simulación: ● trayectorias.csv — columnas: tick, particula_id, x, y, vx, vy

		• energia.csv — columnas: tick, energia_cinetica, energia_potencial, energia_total • benchmark.csv — columnas: n_particulas, ticks, tiempo_segundos, particulas_por_segundo
10	Visualización en Python	Un script visualizar.py debe leer los CSV generados y producir: animación de trayectorias, gráfico de energía vs. tiempo, heatmap de densidad de posiciones y gráfico de rendimiento (benchmark).
11	Benchmarking de rendimiento	El simulador debe medir y reportar el tiempo de ejecución con clock(), calcular la tasa de procesamiento (partículas×ticks / segundos) y guardarlos en benchmark.csv.

Nota: Elaboración propia.

Requerimientos no funcionales

Tabla 2. Requerimientos no funcionales del sistema.

1	Lenguaje y estándar	C puro, estándar C99/C11, compilado con gcc -Wall -Wextra -O2 -lm.
2	Modularidad	El código debe separarse en al menos cuatro archivos: simulador.c, particula.h, fisicas.c, csv_export.c.
3	Gestión de memoria	Toda memoria dinámica (malloc/calloc) debe validarse y liberarse con free(). Toda apertura de archivo debe validarse con fopen.

4	Escalabilidad	El sistema debe funcionar correctamente con N desde 2 hasta al menos 1000 partículas.
5	Reproducibilidad	La semilla aleatoria (srand) debe poder fijarse por parámetro para reproducir simulaciones.
6	Build system	Debe incluirse un Makefile con los targets: all, clean, profile, optimized.
7	Versionamiento	El proyecto debe versionarse con Git y publicarse en GitHub con historial de commits descriptivos.

Nota: Elaboración propia.

Restricciones del Sistema

- El espacio de simulación es un rectángulo 2D cerrado con dimensiones configurables (ancho × alto).
- Las paredes son perfectamente rígidas con reflexión parametrizable.
- La gravedad actúa únicamente en el eje Y (hacia abajo, positivo).
- Las colisiones entre partículas son siempre elásticas (conservan momento y energía cinética).
- El paso de tiempo dt debe ser suficientemente pequeño para estabilidad numérica (dt < 0.01 s recomendado).

Descripción de las Ecuaciones Físicas

Integración de Movimiento (Método de Euler Semi-Implícito)

Para la actualización del estado de las partículas en cada diferencial de tiempo (dt), o tick, se emplea la integración de Euler Semi-Implícita (o Euler-Cromer). A diferencia del método de Euler hacia adelante, este enfoque actualiza primero la velocidad y luego utiliza esta *nueva* velocidad para calcular la posición, lo cual mejora drásticamente la estabilidad numérica y la conservación de la energía en simulaciones físicas.

Las leyes de Newton establecen que la variación de la velocidad depende de la aceleración, y la variación de la posición depende de la velocidad. La discretización para el algoritmo es la siguiente:

1. Actualización de velocidad:

$$vx(t + dt) = vx(t) + ax(t) \cdot dt$$

$$vy(t + dt) = vy(t) + ay(t) \cdot dt$$

2. Actualización de posición:

- $x(t + dt) = x(t) + vx(t + dt) \cdot dt$
- $y(t + dt) = y(t) + vy(t + dt) \cdot dt$

Gravedad (Aceleración Constante)

La gravedad se modela como una fuerza constante que actúa exclusivamente sobre el eje vertical. Asumiendo una convención de coordenadas orientada a la pantalla (donde el eje Y

crece hacia abajo), la gravedad "g" se aplica como un valor positivo que incrementa la velocidad en Y:

- Aceleración horizontal: $a_x = 0$
- Aceleración vertical: $a_y = g$ (donde $g \approx 9.81$ m/s², valor configurable).

En cada tick, la velocidad vertical se actualiza como:

- $v_y(t + dt) = v_y(t) + g \cdot dt$

Fricción (Amortiguamiento Viscoso)

La fricción modela la resistencia del medio (como el aire o un fluido) y produce una fuerza de arrastre proporcional y opuesta a la velocidad. A nivel cinemático, esto se traduce en una desaceleración exponencial gradual. En el código, se multiplica la velocidad por un factor de atenuación basado en el coeficiente "μ":

- $v_x(t + dt) = v_x(t) \cdot (1 - \mu \cdot dt)$
- $v_y(t + dt) = v_y(t) \cdot (1 - \mu \cdot dt)$

Invariante de consistencia: El coeficiente de fricción μ (medido en s^{-1}) debe cumplir que: $\mu < 1/dt$. Si el factor $(1 - \mu \cdot dt)$ se vuelve negativo, la velocidad invertiría su dirección bruscamente por un error numérico, lo cual es físicamente inválido.

Colisión Elástica entre Partículas

Una colisión elástica perfecta conserva tanto el momento lineal del sistema como la energía cinética total.

Paso 1: Vector normal de colisión (n)

Se calcula el vector unitario que va desde el centro de la partícula 1 (p1) al centro de la partícula 2 (p2):

- $dx = p2.x - p1.x$
- $dy = p2.y - p1.y$
- $d = \sqrt{dx^2 + dy^2}$ ← Distancia entre centros
- $nx = dx / d$
- $ny = dy / d$

Paso 2: Velocidad relativa y filtro de aproximación

Se proyecta la diferencia de velocidades sobre el vector normal para obtener un escalar que indica cuán rápido se acercan:

- $dvx = p1.vx - p2.vx$
- $dvy = p1.vy - p2.vy$
- $vrel_n = dvx \cdot nx + dvy \cdot ny$

Condición crítica: Si $vrel_n \leq 0$, las partículas se están alejando mutuamente o se mueven en paralelo. La colisión se ignora para prevenir resoluciones dobles.

Paso 3: Cálculo y aplicación del impulso de colisión (J)

Con masas m_1 y m_2, el impulso escalar bidireccional se define como:

- $J = (2 \cdot m_1 \cdot m_2 \cdot vrel_n) / (m_1 + m_2)$

Se aplica este impulso a las velocidades, respetando la Tercera Ley de Newton (acción y reacción):

- $p1.vx = p1.vx - (J / m_1) \cdot nx$
- $p1.vy = p1.vy - (J / m_1) \cdot ny$
- $p2.vx = p2.vx + (J / m_2) \cdot nx$
- $p2.vy = p2.vy + (J / m_2) \cdot ny$

Paso 4: Corrección de solapamiento ponderada por masa

Para evitar que las partículas queden intersecadas por la naturaleza discreta del tiempo "dt", deben separarse distribuyendo la distancia de superposición según la inercia de cada una (la masa menor se desplaza más):

- $solapamiento = (p1.radio + p2.radio) - d$
- $factor_1 = m_2 / (m_1 + m_2)$
- $factor_2 = m_1 / (m_1 + m_2)$
- $p1.x = p1.x - (solapamiento \cdot factor_1 \cdot nx)$
- $p1.y = p1.y - (solapamiento \cdot factor_1 \cdot ny)$
- $p2.x = p2.x + (solapamiento \cdot factor_2 \cdot nx)$
- $p2.y = p2.y + (solapamiento \cdot factor_2 \cdot ny)$

Colisión con Paredes (Reflexión y Restitución)

Los límites del espacio simulan muros inamovibles. El coeficiente de restitución "e" (donde $0 \leq e \leq 1$) define qué fracción de la velocidad se conserva tras el impacto.

Eje X (Paredes izquierda/derecha):

- Si $(x - radio \leq 0)$:

$vx = |vx| \cdot e$ (Reposicionar a: $x = radio$)

- Si ($x + radio \geq ancho$):

$vx = -|vx| \cdot e$ (Reposicionar a: $x = ancho - radio$)

Eje Y (Paredes superior/inferior):

- Si ($y - radio \leq 0$):

$vy = |vy| \cdot e$ (Reposicionar a: $y = radio$)

- Si ($y + radio \geq alto$):

$vy = -|vy| \cdot e$ (Reposicionar a: $y = alto - radio$)

Análisis de Energía del Sistema

Para auditar la estabilidad de la simulación, se monitoriza la energía mecánica total del sistema, denotada por el símbolo de sumatoria (Σ) sobre todas las partículas (i).

Energía Cinética (Ek): Dependiente de la masa y la magnitud al cuadrado de la velocidad.

- $Ek_total = \Sigma \left[\frac{1}{2} \cdot m_i \cdot (vx_i^2 + vy_i^2) \right]$

Energía Potencial Gravitacional (Ep): Dado que la convención del renderizado define el eje Y creciente hacia abajo, la altura real respecto al "suelo" de la pantalla es ($alto - y$).

- $Ep_total = \Sigma \left[m_i \cdot g \cdot (alto - y_i) \right]$

Energía Mecánica Total (Et):

- $Et = Ek_total + Ep_total$

Principio de Conservación: Si el sistema se configura sin fricción ($\mu = 0$) y con coeficientes de restitución perfectamente elásticos ($e = 1$), la energía mecánica Et debe permanecer constante. Las fluctuaciones menores en el CSV exportado corresponderán estrictamente a los errores de truncamiento del punto flotante en C.

Diseño de las Estructuras y Algoritmos de Simulación

Variables Adicionales

Además de los datos propios de cada partícula, el sistema requiere un conjunto de variables globales que definen el entorno físico, el comportamiento dinámico y las condiciones numéricas de la simulación. Estas variables deben ser tratadas como parte del input del sistema, permitiendo modificar el comportamiento sin alterar el código fuente.

Parámetros físicos del sistema

Estos parámetros controlan las fuerzas y la naturaleza de las colisiones:

- **g (gravedad):**
  - Define la aceleración constante aplicada en el eje vertical.
  - Su signo depende del sistema de coordenadas adoptado.
- **mu (fricción):**
  - Modela la pérdida de energía en el sistema.
  - Afecta directamente la velocidad de las partículas en cada iteración.
- **e (restitución):**
  - Determina qué tan elásticas son las colisiones.

Parámetros numéricos de simulación

Definen la resolución temporal del sistema:

- **dt (delta de tiempo):**
  - Controla la precisión del método de integración.

- Controla que tan “rápido” se verán los cambios en el modelo.

- **steps:** Cantidad total de iteraciones del sistema.

Parámetros del dominio espacial

Definen el entorno físico en el que se mueven las partículas:

double width; // ancho del sistema

double height; // alto del sistema

- Delimitan el espacio de simulación.
- Se utilizan para detectar colisiones con paredes.

Diseño del Struct de Partícula

Dado que el requerimiento funcional (1 y 9) exige la serialización a CSV con datos específicos y cálculos de física 2D continuos (con precisión de punto flotante), el struct en C debe contener atributos espaciales, cinemáticos e identificadores. Se emplea el uso de double en vez de float para minimizar el error de truncamiento para cálculos al momento de realizar cálculos físicos.

A continuación se muestra el respectivo código en el lenguaje de programación C:

```
struct Particula {
    int id;           // Identificador único (Requerimiento 9)
particula_id)
    double x;         // Posición en el eje X
    double y;         // Posición en el eje Y
    double vx;        // Velocidad en el eje X
```

```
    double vy;        // Velocidad en el eje Y
    double masa;      // Masa de la partícula (para colisiones y energía)
    double radio;     // Radio de la partícula (para colisiones y
visualización)
};
```

Descripción de los Atributos

Cada atributo de la estructura tiene un propósito matemático y físico específico:

- **Identidad (int id):** Fundamental para el Requerimiento 9. Permite que el script de Python trace la ruta individual de una partícula a lo largo del tiempo agrupando las filas del archivo trayectorias.csv por este campo.
- **Vector de Posición (double x, double y):** Define el centroide geométrico de la partícula en el espacio bidimensional. Es el estado espacial actualizado en la Fase 2 del método de Euler Semi-Implícito: $x(t + dt) = x(t) + vx(t + dt)$. Además, la "coordenada y" es necesaria para calcular la energía potencial gravitacional. Se especifica en metros (m).
- **Vector de Velocidad (double vx, double vy):** Representa la tasa de cambio de la posición. Son las variables más volátiles del sistema. Son modificadas directamente por la gravedad, la fricción viscosa y los impulsos resultantes de las colisiones. Son indispensables para el cálculo de la energía cinética: $Ek = ½ \cdot m_i \cdot (vx_i^2 + vy_i^2)$. Se especifica en metros por segundo (m/s).
- **Propiedades Inerciales (double masa):** Modela la inercia (resistencia al cambio de movimiento). Interviene directamente en el cálculo del impulso de colisión escalar bidireccional y asegura que, durante la corrección de solapamiento, las partículas más pesadas se

desplacen menos que las más ligeras (conservación del momento). Influye también en proporcionalidad directa con el cálculo de la energía cinética. Se especifica en kilogramos (kg).

- **Límites Físicos (double radio):** Define la dimensión de la partícula. Utilizado en la detección de colisiones, donde el impacto ocurre si la distancia euclidiana entre dos centros es menor a la suma de sus respectivos radios. También determina el punto exacto de rebote contra los bordes del sistema. Se especifica en metros (m).

Bucle de Simulación

El algoritmo debe respetar estrictamente el método de **Euler Semi-Implícito** descrito. El orden de las operaciones se da de la siguiente forma: primero se alteran las velocidades (Fase de fuerzas), luego se actualizan las posiciones, y finalmente se resuelven las restricciones espaciales (colisiones).

Flujo lógico por cada tick:

1. **Fase de Fuerzas (Actualización de Velocidades y Posiciones):** Se itera por cada una de las partículas y se realiza lo siguiente secuencialmente:
    - Aplicar gravedad: Adicionar a vy sumando g * dt.

      $vy \leftarrow vy + g*dt$
    - Aplicar amortiguamiento viscoso (fricción): Multiplicar vx y vy por (1 - μ * dt).

      $vy \leftarrow vy * (1- \mu * dt)$
    - Actualizar posición x sumando vx * dt.

      $x \leftarrow x + vx * dt$
    - Actualizar posición y sumando vy * dt.

      $y \leftarrow y + vy * dt$

2. **Detección y Resolución de Colisiones (Partícula - Pared):** Se itera por cada una de las partículas y se realiza lo siguiente secuencialmente:
    - Comprobar límites (0, ancho) para X y (0, alto) para Y considerando el radio.
    - Si hay impacto: aplicar el coeficiente de restitución e invertir la velocidad correspondiente. Además, efectuar la corrección de posición para que no se "escape" del límite.
3. **Detección y Resolución de Colisiones (Partícula - Partícula):** Se itera sobre cada par único de partículas y se realiza lo siguiente secuencialmente:
    - Calcular la distancia euclidiana entre centros.
    - Si distancia < (radio1 + radio2), calcular la velocidad relativa normal.
    - Si vrel_n > 0 (se están acercando), calcular el impulso J y aplicar los cambios a las velocidades.
    - Aplicar la corrección de solapamiento ponderada por masa.
4. **Fase de Cálculo de Datos.**
    - Calcular energía cinética y potencial total del sistema.
    - Registrar el estado actual en los archivos CSV (trayectorias y energía).

A continuación se muestra una primera aproximación del algoritmo en el lenguaje C:

```
// Bucle iterando hasta alcanzar el tiempo total de simulación

for (int tick = 0; tick < TICKS_TOTALES; tick++) {

    // FASE 1: Integración (Fuerzas -> Velocidad -> Posición)
```

```
    for (int i = 0; i < N; i++) {

        // 1.1 Aplicar Gravedad (Aceleración Constante en Y)

        sistema[i].vy += g * dt;

        // 1.2 Aplicar Fricción (Amortiguamiento Viscoso)

        // Se asegura que (1 - mu * dt) > 0 por restricción del sistema

        double factor_friccion = 1.0 - (mu * dt);

        sistema[i].vx *= factor_friccion;

        sistema[i].vy *= factor_friccion;



        // 1.3 Actualizar Posición con la NUEVA velocidad

        sistema[i].x += sistema[i].vx * dt;

        sistema[i].y += sistema[i].vy * dt;

    }

    // FASE 2: Detección y Resolución de Colisiones (Paredes)
```

```
    for (int i = 0; i < N; i++) {

        // Lógica de rebote y reposición para evitar que la partícula

        // quede "atrapada" fuera del límite debido al paso dt.

        // Ejemplo eje X:

        if (sistema[i].x - sistema[i].radio <= 0) {

            sistema[i].x = sistema[i].radio;

            sistema[i].vx = fabs(sistema[i].vx) * e;

        } else if (sistema[i].x + sistema[i].radio >= ancho) {

            sistema[i].x = ancho - sistema[i].radio;

            sistema[i].vx = -fabs(sistema[i].vx) * e;

        }

        // Repetir lógica análoga para el eje Y

    }

    // FASE 3: Detección y Resolución de Colisiones (Partícula-Partícula)
```

```
    resolver_colisiones_particulas(sistema, N);

    // FASE 4: Cálculo de Datos

    EnergiaSistema energia = calcular_energia(sistema, N, g, alto);

    escribir_csv_trayectorias(archivo_tray, sistema, N, tick);

    escribir_csv_energia(archivo_energia, energia, tick);

}
```

Análisis algorítmico

- **Fase 1:** O(5N) = O(N) Cinco operaciones constantes por partícula (2 sumas, 3 multiplicaciones), iteradas N veces.
- **Fase 2:** O(4N) = O(N) Cuatro comparaciones de límites por partícula, iteradas N veces.
- **Fase 3:** $O(N \cdot (N-1)/2) = O((N^2-N)/2) = O(N^2)$ Se elimina el término $-N$ por ser de menor orden, y la constante 1/2 desaparece en notación Big-O.
- **Fase 4:** O(2N) = O(N) Una pasada para calcular energías y otra para escribir el CSV, ambas sobre N partículas.

$O(5N) + O(4N) + O(N \cdot (N-1)/2) + O(2N) = O(11N + (N^2-N)/2) = O(11N + N^2/2 - N/2) = O(N^2/2 + 10.5N) =$ **$O(N^2)$**

Se elimina el término 10.5N por ser de menor orden que N^2, y la constante 1/2 desaparece, quedando **$O(N^2)$** por tick y **$O(T \cdot N^2)$** en total.

Referencias

Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, 9(3), 90–95.

Kernighan, B. W., & Ritchie, D. M. (1991). *El lenguaje de programación C*. Pearson Educación.

Kleppner, D., & Kolenkow, R. (2014). *An introduction to mechanics (2nd ed.)*. Cambridge University Press.

Press, W. H. et al. (2007). *Numerical Recipes in C: The Art of Scientific Computing* (3rd Ed.). Cambridge University Press.

Teschner, M. et al. (2003). *Optimized Spatial Hashing for Collision Detection of Deformable Objects*. Proceedings of VMV 2003.